

## Skill 5

1) Task 1: To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases.

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main() {
```

```
    char input[200];
```

```
    char result[200];
```

```
    printf("Enter text inside single quotes: ");
```

```
    fgets(input, sizeof(input), stdin);
```

```
    // Remove newline
```

```
    input[strcspn(input, "\n")] = '\0';
```

```
    // Validate single quotes
```

```
    int len = strlen(input);
```

```
    if (len >= 2 && input[0] == '\'' && input[len - 1] == '\') {
```

```
        // Store content without the single quotes
```

```
        strcpy(result, input + 1);
```

```
        result[len - 2] = '\0';
```

```
printf("\nOriginal input: %s\n", input);

printf("Parsed content: %s\n", result);


printf("Single quotes preserve the content literally.\n");
printf("Variable expansion is ignored.\n");
printf("Quoted string stored successfully.\n");
printf("Parsing result: VALID\n");


} else {

    printf("\nInvalid input!\n");

    printf("Please enter the text inside single quotes.\n");

    printf("Example: 'Hello $USER'\n");

    printf("Parsing result: INVALID\n");

}

return 0;
}
```

```
ashwithreddy_earla@AshwithReddy:~$ nano skill5.c
ashwithreddy_earla@AshwithReddy:~$ gcc skill5.c -o skill5
ashwithreddy_earla@AshwithReddy:~$ ./skill5
Enter text inside single quotes: 'Hello World'

Original input: 'Hello World'
Parsed content: Hello World
Single quotes preserve the content literally.
Variable expansion is ignored.
Quoted string stored successfully.
Parsing result: VALID
ashwithreddy_earla@AshwithReddy:~$ |
```

2) Task 2: To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#include <stdlib.h>
```

```
int main() {
```

```
    char input[200];
```

```
    char result[200];
```

```
    printf("Enter text inside double quotes: ");
```

```
    fgets(input, sizeof(input), stdin);
```

```
    // Remove newline
```

```
    input[strcspn(input, "\n")] = '\0';
```

```
    int len = strlen(input);
```

```
    // Validate double quotes
```

```
    if (len >= 2 && input[0] == '"' && input[len - 1] == '"') {
```

```
        // Remove double quotes
```

```
        strcpy(result, input + 1);
```

```
        result[len - 2] = '\0';
```

```
printf("\nOriginal input: %s\n", input);

printf("Parsed content: %s\n", result);


printf("Double quotes preserve spaces.\n");


// Demonstrate variable expansion

char *user = getenv("USER");

if (strstr(result, "$USER") != NULL && user != NULL) {

    char expanded[200];

    // Simple $USER expansion

    char *position = strstr(result, "$USER");

    int before = position - result;

    strncpy(expanded, result, before);

    expanded[before] = '\0';

    strcat(expanded, user);

    strcat(expanded, position + 5);

    printf("After variable expansion: %s\n", expanded);

} else {

    printf("No variable found for expansion.\n");
```

```

    }

    printf("Nested tokens are preserved inside double quotes.\n");

    printf("Output validation: VALID\n");

    printf("Quoted command test: PASSED\n");

} else {

    printf("\nInvalid input!\n");

    printf("Please use double quotes.\n");

    printf("Example: \"Hello World\"\n");

    printf("Output validation: INVALID\n");

    printf("Quoted command test: FAILED\n");

}

return 0;

}

```

```

ashwithreddy_earla@AshwithReddy:~$ ^C
ashwithreddy_earla@AshwithReddy:~$ nano skill5.2.c
ashwithreddy_earla@AshwithReddy:~$ gcc skill5.2.c -o skill5.2
ashwithreddy_earla@AshwithReddy:~$ ./skill5.2
Enter text inside double quotes: "Hello World $USER Test"

Invalid input!
Please use double quotes.
Example: "Hello World"
Output validation: INVALID
Quoted command test: FAILED
ashwithreddy_earla@AshwithReddy:~$ |

```